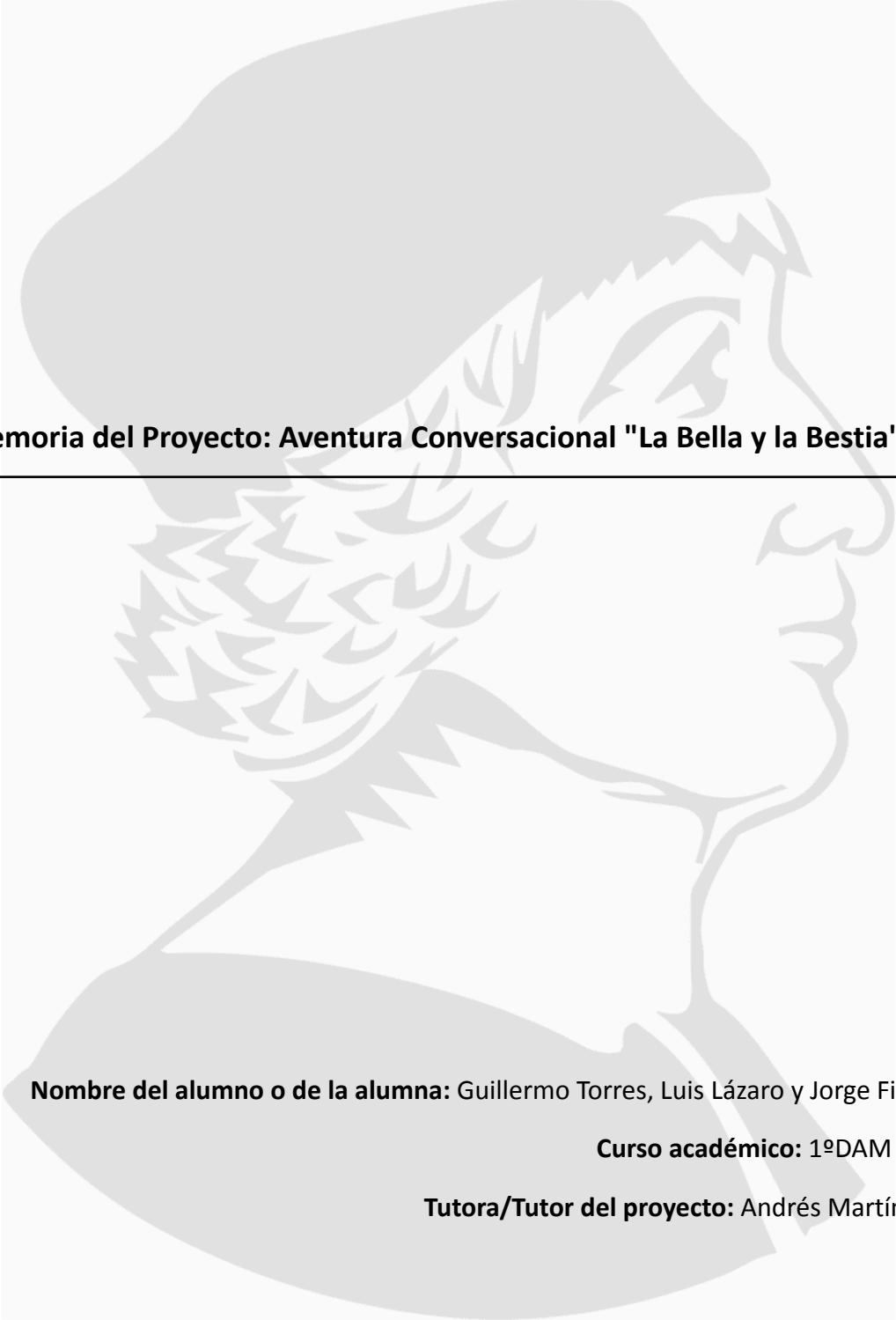


PORTADA



Memoria del Proyecto: Aventura Conversacional "La Bella y la Bestia"

Nombre del alumno o de la alumna: Guillermo Torres, Luis Lázaro y Jorge Fillol

Curso académico: 1ºDAM G2

Tutora/Tutor del proyecto: Andrés Martínez

ÍNDICE

1. INTRODUCCIÓN Y JUSTIFICACIÓN	3
• 1.1. Contexto del Desarrollo	
• 1.2. Justificación de la Temática	
2. OBJETIVOS DEL PROYECTO	5
• 2.1. Objetivo General	
• 2.2. Objetivos Específicos y Técnicos	
3. METODOLOGÍA Y HERRAMIENTAS	6
• 3.1. Planificación y Diseño	
• 3.2. Entorno de Desarrollo	
4. ANÁLISIS TÉCNICO Y ARQUITECTURA DEL CÓDIGO	7
• 4.1. Librerías y Constantes Visuales	
• 4.2. Sistema de Autenticación (Login)	
• 4.3. Selección de Personaje y Configuración Inicial	
• 4.4. Mecánicas de Juego por Ruta (Deep Dive)	
○ A. Ruta de Bella: Puzzles Lógicos y Combate Dialéctico	
○ B. Ruta de Gastón: Simulación de Gestión y RPG	
○ C. Ruta de la Bestia: Redención y Métodos Auxiliares	
• 4.5. Validación de Datos y Manejo de Excepciones	
• 4.6. Algoritmia de Combate y Cálculo de Daño	
• 4.7. Control de Tiempo y Experiencia de Usuario (UX)	
5. PRUEBAS, VALIDACIÓN Y MANEJO DE ERRORES	10
• 5.1. Validación de Entradas (Input Handling)	
• 5.2. Casos de Uso Verificados	
• 5.3. Pruebas de Valores Límite (Boundary Testing)	
• 5.4. Detección y Corrección de Bucles Infinitos	
6. DIAGRAMA (E/R) Y CASOS DE USO	15
7. CONCLUSIONES Y VALORACIÓN PERSONAL	18
• 7.1. Líneas de Trabajo Futuro (Mejoras Potenciales)	
8. BIBLIOGRAFÍA Y RECURSOS	19

1. INTRODUCCIÓN Y JUSTIFICACIÓN

1.1. Contexto del Desarrollo

El presente proyecto consiste en el desarrollo integral de un videojuego de consola programado en el lenguaje **Java**, diseñado bajo el género de "Aventura Conversacional" (estilo *Elige tu propia aventura*). Este trabajo se enmarca dentro del módulo de Programación del ciclo de **1º de Desarrollo de Aplicaciones Multiplataforma (DAM)**, sirviendo como proyecto final para consolidar las competencias adquiridas durante el trimestre.

La aplicación recrea una experiencia narrativa interactiva donde el usuario no es un mero espectador, sino el protagonista activo de la historia. A nivel técnico, el software se ejecuta en un entorno de consola y se estructura en un único archivo de código fuente (*bellaBestia2.java*), integrando bibliotecas fundamentales como `java.util.Scanner` para la entrada de datos y `java.util.Random` para la generación de eventos aleatorios.

El propósito fundamental de este software va más allá del entretenimiento; busca integrar y demostrar el dominio de conocimientos teóricos y prácticos esenciales, tales como:

- **El uso de estructuras de control:** Implementación de condicionales (`if`, `switch`) para ramificar la historia y bucles (`while`, `do-while`) para gestionar menús y sistemas de combate.
- **Gestión de estado:** Manejo de variables para controlar puntos de vida, inventario y progreso en la aventura.
- **Validación de datos:** Desarrollo de sistemas robustos de manejo de errores para evitar cierres inesperados ante entradas de usuario incorrectas.
- **Fomento de la autonomía:** Resolución de problemas lógicos complejos y capacidad de planificación previa mediante diagramas de flujo.

1.2. Justificación de la Temática

Se ha seleccionado el cuento clásico de "**La Bella y la Bestia**" como núcleo narrativo del proyecto. Esta elección no es meramente estética, sino funcional y estratégica: la obra original presenta personajes con arquetipos muy definidos y conflictos internos claros, lo que la convierte en el material ideal para estructurar una aventura ramificada con múltiples decisiones y finales.

La adaptación al formato de "Elige tu propia aventura" permite explotar las siguientes dimensiones:

1. **Perspectiva Múltiple:** A diferencia de una novela lineal, este software permite al usuario encarnar a tres roles distintos, cada uno con mecánicas de juego únicas que reflejan su personalidad:
 - **Bella:** Su ruta se centra en el intelecto, la resolución de acertijos y la empatía.

- **La Bestia:** Su jugabilidad explora la gestión de la ira, la fuerza bruta y la redención contrarreloj.
 - **Gastón:** Introduce mecánicas de gestión de recursos (economía y caza) y combate ofensivo.
2. **Rejugabilidad y Profundidad:** La estructura del juego fomenta que el usuario explore diferentes caminos para descubrir todos los desenlaces posibles, desde finales trágicos hasta resoluciones heroicas o secretas.
3. **Complejidad Lógica:** Las motivaciones opuestas de los personajes (intelecto vs. fuerza, ego vs. sacrificio) facilitaron el diseño de árboles de decisión complejos, obligando a implementar una lógica de programación variada que incluye minijuegos (como "Piedra, Papel o Tijera" o adivinanzas matemáticas) integrados orgánicamente en la trama.

```
//TÍTULO CON ARTE ASCII Y COLOR
System.out.println(YELLOW + "-----" + RESET);
System.out.println(RED + " . . . : : LA ROSA ENCANTADA : : . . . " + RESET);
System.out.println(YELLOW + "-----" + RESET);
System.out.println(RED + " (@) " + RESET);
System.out.println(GREEN + " | " + RESET);
System.out.println(GREEN + " \\|/ " + RESET);
System.out.println(GREEN + " | " + RESET);
System.out.println(x: " BIENVENIDO A LA BELLA Y LA BESTIA");
System.out.println(CYAN + " Aventura Conversacional DAM" + RESET);
System.out.println(YELLOW + "-----\n" + RESET);
```

1. Imagen ASCII del principio del programa

2. OBJETIVOS DEL PROYECTO

2.1. Objetivo General

El propósito principal es consolidar los conocimientos de programación estructurada mediante la creación de una aplicación compleja que gestione estados, flujos lógicos y validación de datos, ofreciendo al usuario final una experiencia de entretenimiento interactivo y rejugable.

2.2. Objetivos Específicos y Técnicos

Para garantizar la calidad del software, se definieron los siguientes hitos técnicos:

- **Control de Acceso:** Implementar un sistema de seguridad "login" básico mediante contraseña.
- **Narrativa Ramificada (Branching Logic):** Estructurar el código para soportar tres líneas temporales paralelas que no se cruzan a nivel de ejecución, pero comparten el mismo universo narrativo.
- **Persistencia de Datos en Ejecución:** Gestionar variables de estado vitales como puntos de vida (PV), inventario (items), dinero y moralidad ("Bondad").
- **Aleatoriedad Controlada (RNG):** Utilizar la clase Random de Java para que los combates y minijuegos no sean deterministas, aumentando la rejugabilidad.
- **Robustez y UX:** Asegurar que el programa no se cierre abruptamente (crash) ante entradas de usuario inválidas, utilizando bucles de validación y limpieza de buffer (Scanner).
- **Modularidad y Reutilización de Código:** Descomponer la lógica del juego en funciones o métodos independientes (para combates, menús, tiendas) con el fin de reducir la duplicidad de código y facilitar la depuración y el mantenimiento del software.
- **Gestión de Estructuras de Datos Estáticas:** Implementar el uso de Arreglos (Arrays) o Matrices para gestionar colecciones de datos finitos, como el catálogo de la tienda, la lista de enemigos disponibles o el inventario limitado del jugador.
- **Formato y Legibilidad de Salida (UI):** Diseñar una interfaz de consola clara mediante el uso de separadores visuales y espaciados
- **Balanceo de Mecánicas (Game Design):** Calibrar las fórmulas matemáticas de daño, costes de objetos y recompensas para asegurar una curva de dificultad progresiva, evitando que el juego sea imposible de completar o trivialmente fácil.



3. METODOLOGÍA Y HERRAMIENTAS

3.1. Planificación y Diseño

Antes de la escritura del código, se realizó una fase de ingeniería de software utilizando diagramas de flujo en la herramienta Draw.io. Estos diagramas actuaron como "mapas" lógicos para visualizar:

1. Los puntos de bifurcación (decisiones del jugador).
 2. Los bucles de juego (combates y menús recurrentes).
 3. Las condiciones de victoria y derrota (Game Over).
- **Guionización Narrativa:** Se elaboró un borrador de texto (script) independiente para redactar los diálogos, las descripciones de escenarios y las respuestas de los NPCs, asegurando la coherencia argumental de las tres líneas temporales antes de su implementación mediante sentencias de salida.
 - **Pseudocódigo y Lógica Modular:** Se esquematizaron las funciones principales (como el sistema de tienda o la mecánica de combate) en pseudocódigo para validar la lógica de los bucles while y las estructuras condicionales if-else antes de la traducción final a sintaxis Java.

3.2. Entorno de Desarrollo

La implementación se llevó a cabo utilizando el entorno de desarrollo integrado (IDE) Eclipse, que facilitó la depuración de errores sintácticos y la compilación del archivo único bellaBestia2.java. Se utilizaron herramientas de IA como ChatGPT y Gemini para consultas puntuales sobre optimización de lógica y sintaxis.

Kit de Desarrollo (JDK): El proyecto se compiló bajo el estándar de **Java Development Kit (JDK)**, garantizando la compatibilidad de las estructuras de control utilizadas.

Bibliotecas Estándar: Se limitó el uso de dependencias externas exclusivamente a las librerías nativas del paquete java.util, lo que asegura la portabilidad del código y permite su ejecución en cualquier máquina con la máquina virtual de Java (JVM) instalada sin necesidad de configuraciones adicionales.



4. ANÁLISIS TÉCNICO Y ARQUITECTURA DEL CÓDIGO

El núcleo del proyecto reside en el archivo `bellaBestia2.java`. A continuación, se detalla su funcionamiento interno desglosado por bloques lógicos.

4.1. Librerías y Constantes Visuales

El programa importa `java.util.Scanner` para la entrada de datos y `java.util.Random` para la generación de números aleatorios. Una característica destacada es la definición de constantes de escape ANSI (ej. `String RED = "\u001B[31m";`), lo que permite renderizar texto en color en la consola, asignando una identidad visual a cada personaje (Amarillo para Bella, Rojo para Bestia, etc.).

4.2. Sistema de Autenticación (Login)

Al inicio del método `main`, se implementa un bucle `do-while` que actúa como barrera de entrada. El sistema solicita la clave "jugones". Mientras la entrada no coincida exactamente con este String (validado mediante `.equals()`), el programa impide el acceso al menú principal, cumpliendo con el requisito de seguridad.

4.3. Selección de Personaje y Configuración Inicial

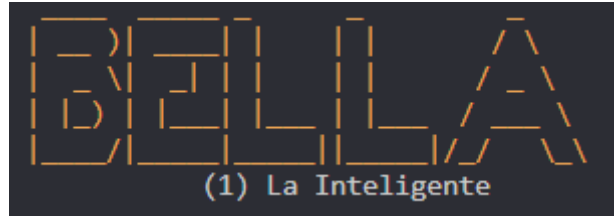
El menú principal utiliza arte ASCII para presentar a los tres protagonistas. La selección se gestiona mediante una estructura de control `switch`, que inicializa las variables base según el rol elegido:

- **Bella:** Se inicializa con **70 PV** (Puntos de Vida). Perfil: Inteligencia y diplomacia.
- **Bestia:** Se inicializa con **120 PV**. Perfil: Fuerza bruta y resistencia.
- **Gastón:** Se inicializa con **100 PV**. Perfil: Combate equilibrado y gestión de recursos.

--Elige el personaje para tu aventura--

4.4. Mecánicas de Juego por Ruta (Deep Dive)

A. Ruta de Bella: Puzzles Lógicos y Combate Dialéctico



Esta ruta evita la fuerza física. El código implementa una serie de desafíos secuenciales:

1. **El Acertijo de Ding Dong:** Un condicional if evalúa si la respuesta del usuario contiene la palabra "reloj". El éxito otorga puntos a la variable bondad; el fallo reduce la variable "petalosRestantes".

```
=====
CAPÍTULO 1: LA PRISIONERA
=====
Te encuentras encerrada en una habitación lujosa pero fría.
Tienes hambre y estás asustada. Escuchas un ruido en la puerta.
=====

( 12 )
| / |  <- [DING DONG]

Un reloj parlante 'Ding Dong' te impide el paso.
Ding Dong: 'Señorita, no puede salir... a menos que demuestre intelecto.'
Ding Dong: 'Responde: Tengo agujas pero no coso, tengo números pero no cuento. ¿Qué soy?'
[+] Tu respuesta: 
```

2. **Minijuego de Chip (Adivinanza):** Se genera un número aleatorio `rand.nextInt(10) + 1`. El jugador entra en un bucle while con 3 intentos. El feedback es dinámico ("Es más alto", "Es más bajo"). Ganar cura al jugador (+30 PV).

```
( )
| |  <- [CHIP]
|_|

Una pequeña taza 'Chip' se acerca saltando.
Chip: '¡Mamá! ¡Hay una chica! ¡Juguemos a adivinar!'
Sra. Potts: 'Si ganas a Chip, te daré una sopa caliente para recuperar fuerzas.'

[MINIJUEGO]: Adivina el número de Chip (1-10). Tienes 3 intentos.
>> Introduce un número: 
```


3. **Piedra, Papel o Tijera (Lumière):** Implementado con lógica condicional anidada. Ganar desbloquea el booleano tieneEspejo = true, un objeto clave para el final secreto.

```
En la biblioteca encuentras a Lumiere (el candelabro).
|
-O-
|   <- [LUMIÈRE]
Lumiere: 'Mon ami! Si me ganas a Piedra, Papel o Tijera, te daré un objeto especial.'
1. Piedra | 2. Papel | 3. Tijera
```

4. **Combate Final (La Bestia):** Innovación en el sistema de combate. Los "ataques" son opciones de diálogo ("Argumentar con Lógica", "Mirada Compasiva"). El código calcula la probabilidad de éxito (Impacto Total, Parcial o Esquiva) usando porcentajes aleatorios.

```
=====
CLÍMAX: EL ENCUENTRO CON LA BESTIA
=====
(o o)
( -- ) <- [LA BESTIA]
/|  \
La Bestia aparece, furiosa porque estás fuera de tu cuarto.
Bestia: '¿QUÉ HACES AQUÍ?!'
No puedes escapar. ¡Debes calmarlo o luchar!
```

B. Ruta de Gastón: Simulación de Gestión y RPG

```

GASTON
(3) El Cazador
```

Es la ruta más compleja a nivel de código debido a la gestión de inventario y economía.

- **Ciclo Temporal:** Un bucle while controla la variable tiempoDia. El jugador tiene un límite de 8 "horas" (turnos) para prepararse antes del asalto final.

```
[+] Has elegido a Gastón

=====
CAPÍTULO 1: EL EGO DEL CAZADOR
=====
Despiertas sintiéndote genial. Nadie en el pueblo es tan fuerte como tú.
Tu objetivo: Encontrar a Bella y demostrar que eres el mejor.
=====

[HORA]: 8:00
Dinero: 5 monedas | Vida: 100
¿Qué quieres hacer?
1. Ir a la Biblioteca (Buscar a Bella)
2. Ir al Mercado (Comprar/Vender)
3. Ir a la Taberna (Lefou y Pueblo)
4. Ir al Bosque (Cazar animales)
5. Finalizar el día (Ir al rescate)
[+] Elige:
```



- **Sistema de Mercado:** Gastón puede cazar animales (RNG para determinar presa: conejo, ciervo, jabalí) y vender los ítems obtenidos. El código calcula las ganancias: $(carneJabali * 15) + (cuernoCiervo * 12)$... y actualiza la variable dinero.

```
Te adentras en el bosque...
¡Un JABALÍ salvaje!
1. Atacar
1
El jabalí embiste.
1. Atacar
1
El jabalí embiste.
1. Atacar
1
Jabalí muerto.

[HORA]: 9:00
Dinero: 5 monedas | Vida: 80
¿Qué quieres hacer?
1. Ir a la Biblioteca (Buscar a Bella)
2. Ir al Mercado (Comprar/Vender)
3. Ir a la Taberna (Lefou y Pueblo)
4. Ir al Bosque (Cazar animales)
5. Finalizar el día (Ir al rescate)
[+] Elige: 2
--- MERCADO ---
1. Vender Caza | 2. Tienda Misteriosa | 3. Salir
1
Has vendido todo por 15 monedas.
```

- **Tienda y Objetos:** Con el dinero acumulado, se pueden comprar mejoras como la "Punta Sigilosa" (aumenta probabilidad de caza) o pociones.
- **Boss Fight (Lobo Alfa):** Un combate opcional que otorga la segunda mitad del mapa, necesaria para el final "bueno" de Gastón.

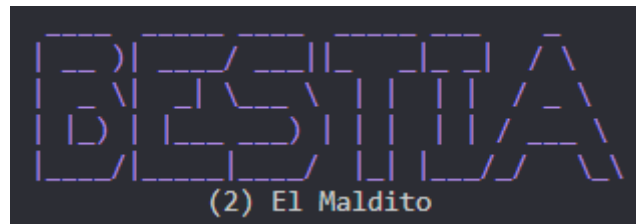
```
} else if (accion == 4) {
    // BOSQUE (Caza y Combate)
    System.out.println(GREEN + "Te adentras en el bosque..." + RESET);

    if (leFouAmigo && !mapaParte2) {
        // JEFE LOBO (Boss de zona)
        System.out.println(RED + "¡Un LOBO ALFA protege el Santuario!" + RESET);
        int vidaLobo = 80;
        while(vidaJugador > 0 && vidaLobo > 0){
            System.out.println("Tu vida: " + vidaJugador + " | Lobo: " + vidaLobo);
            System.out.println(x: "1. Atacar | 2. Poción");
            int act = sc.nextInt();
            if(act==1) { vidaLobo -= 20; System.out.println(x: "Golpeas al lobo."); }
            else if (act==2 && pocion) { vidaLobo -= 50; System.out.println(PURPLE + "¡Usas poder oscuro!" + RESET); pocion=false; }

            if(vidaLobo>0) {
                vidaJugador -= 15;
                System.out.println(RED + "El lobo te muerde (-15 PV)" + RESET);
            }
        }
        if(vidaJugador > 0){
            mapaParte2 = true;
            System.out.println(PURPLE + "¡Has encontrado la OTRA MITAD DEL MAPA en el santuario!" + RESET);
        }
    }
}
```



C. Ruta de la Bestia: Redención y Métodos Auxiliares



Esta ruta introduce la mecánica de la moralidad y el uso de métodos externos.

```
=====
CAPÍTULO 1: LA MALDICIÓN DEL PRÍNCIPE
=====
Despiertas en tu forma de Bestia, solo en el gran salón del castillo.
La rosa encantada pierde lentamente sus pétalos. Debes decidir cómo actuar para romper la maldición.

===== MENÚ DE LA BESTIA =====
1. Reunirte con los sirvientes (Lumière y Ding Dong)
2. Salir a patrullar el bosque
3. Buscar a la Bruja en el claro
4. Prepararte para la llegada de Gastón
5. Ver tu estado actual
6. Abandonar el castillo (final alternativo)
=====
[+] Elige una opción: 
```

- **Gestión de Tiempo:** Se controla mediante un contador de acciones. Si llega a 12 sin resolver la trama, se fuerza un final negativo.
- **El Ritual (Método realizarRitual):** Para no sobrecargar el main, se extrajo la lógica del final a un método estático independiente. Este método plantea tres dilemas éticos al jugador. Se requiere un contador de aciertos ≥ 2 para romper la maldición y obtener el final "Humano por Amor".

```
La rosa brilla intensamente. Sientes que puedes intentar un ritual para romper la maldición.
¿Quieres intentar el ritual final ahora?
1. Sí
2. No
[+] Elige: 1

--- RITUAL PARA ROMPER LA MALDICIÓN ---
La rosa se ilumina y escuchas una voz que susurra preguntas a tu corazón.
Necesitas al menos 2 respuestas correctas para recuperar tu humanidad.

1) ¿Qué valor importa más para recuperar la humanidad?
1. Poder
2. Amor
3. Orgullo
[+] Elige (1-3): 2

2) ¿A quién perdonarías primero?
1. A ti mismo por tus errores.
2. A quienes se burlaron de ti.
3. A nadie, nadie merece perdón.
[+] Elige (1-3): 1
```

- **Interacción con NPCs:** Variables booleanas como `brujaHostil` y `lumiereAmigo` cambian dinámicamente según las decisiones previas (ej. aceptar o rechazar la poción oscura), desbloqueando nuevas opciones de menú.

```

2) ¿A quién perdonarías primero?
1. A ti mismo por tus errores.
2. A quienes se burlaron de ti.
3. A nadie, nadie merece perdón.
[+] Elige (1-3): 1

3) ¿Aceptarías sacrificios por los que quieres?
1. Sí, incluso si pongo en riesgo mi vida.
2. Solo si no pierdo nada importante.
3. No, primero estoy yo.
[+] Elige (1-3): 1

Has terminado el ritual. Aciertos: 3 de 3.

FINAL: HUMANO POR AMOR

>> Final alcanzado para Bestia: Humano por Amor

--- FIN DEL JUEGO ---

```

4.5. Validación de Datos y Manejo de Excepciones

Dada la naturaleza interactiva de la aplicación en consola, la estabilidad ante entradas erróneas es crítica.

- **Limpieza del Buffer:** Se identificó el problema común del "salto de línea fantasma" al mezclar métodos como `nextInt()` y `nextLine()`. Para solucionarlo, se implementó una limpieza manual del
- `buffer (sc.nextLine())` tras cada lectura numérica, evitando que el programa se salte diálogos o inputs posteriores.
- **Filtrado de Opciones:** En los menús de decisión, se utilizan bucles `while` que envuelven la solicitud de datos. Si el usuario introduce un número fuera del rango esperado (ej. una opción



- "4" en un menú de 3 opciones), el sistema muestra un mensaje de error en color rojo y repite la iteración del bucle, impidiendo el avance hasta obtener un dato válido.

4.6. Algoritmia de Combate y Cálculo de Daño

El sistema de combate no es estático, sino que emplea fórmulas matemáticas para simular variabilidad.

- **Cálculo de Daño Dinámico:** El daño infligido por el jugador o los enemigos se calcula mediante la fórmula:

$$\text{Daño} = \text{AtaqueBase} + (\text{RNG} \times \text{Multiplicador})$$
 Esto asegura que ningún combate sea idéntico al anterior.
- **Sistema de Críticos y Fallos:** Se utiliza la generación de un número entre 1 y 100 para determinar eventos especiales. Por ejemplo, si el valor generado es >90 , se considera un "Golpe Crítico" (doble daño); si es <10 , se considera un "Fallo", narrando que el personaje ha tropezado o el enemigo ha esquivado.

4.7. Control de Tiempo y Experiencia de Usuario (UX)

Para mitigar la rapidez de ejecución nativa de Java y permitir que el usuario lea la narrativa cómodamente:

- **Formato de Salida:** Se estructuró el código para imprimir líneas separadoras (ej. "-----") antes y después de cada bloque de eventos importante, facilitando la legibilidad visual en la consola de comandos.



5. PRUEBAS, VALIDACIÓN Y MANEJO DE ERRORES

5.1. Validación de Entradas (Input Handling)

Uno de los mayores desafíos fue evitar que el programa colapsara (Exception in thread main) si el usuario introducía una letra cuando se esperaba un número.

- **Solución:** Se implementó sistemáticamente la comprobación `if(sc.hasNextInt())` antes de consumir el dato con `sc.nextInt()`.
- **Limpieza de Buffer:** En caso de error, el bloque `else` ejecuta `sc.next()` para descartar la entrada inválida y permitir al usuario intentarlo de nuevo sin reiniciar el juego.

5.2. Casos de Uso Verificados

- **Caso "Elige tu propia aventura":** Se verificó el flujo completo: Inicio -> Contraseña -> Selección -> Juego -> Final. Se comprobó que todas las ramas conducen a un final cerrado (muerte, victoria o final neutro).
- **Caso "Introducir Contraseña":** Se realizaron pruebas de caja negra introduciendo claves erróneas y vacías para asegurar la inviolabilidad del menú principal.

5.3. Pruebas de Valores Límite (Boundary Testing)

Se realizaron pruebas exhaustivas en los puntos críticos donde las variables numéricas alcanzan sus extremos para asegurar que la lógica condicional no fallara:

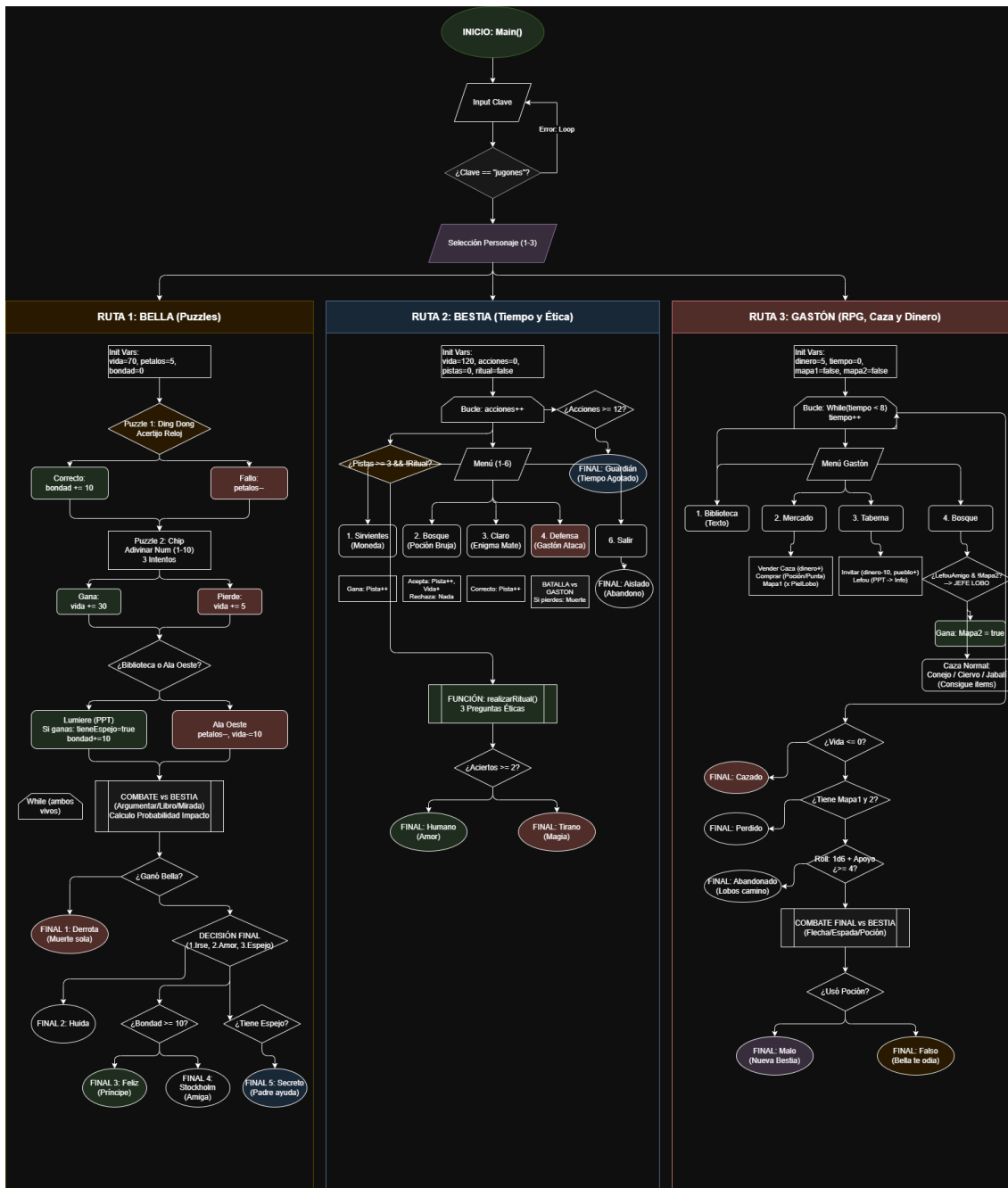
- **Salud Negativa:** Se verificó que, si el daño recibido dejaba los Puntos de Vida (PV) en 0 o en números negativos (ej. -5), el sistema disparara correctamente la condición `if (pv <= 0)` llevando a la pantalla de "Game Over" en lugar de permitir seguir jugando con salud negativa.
- **Economía (Ruta Gastón):** Se intentó comprar objetos en la tienda teniendo exactamente el dinero necesario y teniendo una moneda menos. Esto validó que la condición `if (dinero >= precio)` funcionara correctamente, impidiendo compras con saldo insuficiente y permitiéndolas con saldo exacto.

5.4. Detección y Corrección de Bucles Infinitos

Durante la fase de desarrollo, se identificó un error lógico en el sistema de combate donde, bajo ciertas condiciones aleatorias, el turno del enemigo nunca terminaba.

- **Solución:** Se añadieron trazas de impresión (`System.out.println`) temporales para monitorear el valor de las variables de control en tiempo real. Esto permitió corregir la condición de salida del bucle `while` que gestiona la pelea, asegurando que siempre exista un ganador o un perdedor, evitando bloqueos (soft-locks) de la aplicación.

6. DIAGRAMA (E/R) Y CASOS DE USO



3. DIAGRAMA (E/R) BELLA BESTIA



Casos de uso	Juego elige tu propia historia
Fuentes	El usuario quiere jugar al juego elige tu propia historia, para ello el jugador debe de tener el programa instalado en su ordenador, no alterar el código del programa, ejecutar el juego para poder comenzar y finalmente introducir la contraseña "jugones", para poder comenzar el juego.
Actor	El usuario que elige entre los 3 personajes principales como el que quiera jugar la historia, y el propio programa el cual actúa como los personajes secundarios de la historia.
Descripción	Se trata de una historia sobre "la bella y la bestia" con múltiples finales distintos a parte de el de la historia original, en el que puedes jugar como Bella, la bestia, o como el antagonista de la historia original Gastón. El jugador debe hacer decisiones las cuales afectan a la historia, realizar combates los cuales pueden llevar a distintos finales dependiendo del resultado y completar minijuegos para poder cumplir prerequisites necesarios para poder obtener algunos de los distintos finales.
Flujo básico	1. El usuario abre el programa. 2. El usuario introduce la contraseña "jugones" dentro del programa cuando se lo pide el texto. 3. El usuario elige entre los personajes principales introduciendo un número del 1 al 3. 4. El usuario elige entre las distintas opciones que te da la historia introduciendo el número de la opción que quieres elegir que aparece delante de las distintas opciones que te da el programa. 5. Si entras en un combate dentro del programa, eliges entre los distintos ataques que tiene tu personaje introduciendo el número que aparece delante del ataque que quieres usar hasta que tú o el enemigo pierda. 6. Si entras en un minijuego, sigue las instrucciones que te dará el programa sobre dicho minijuego para intentar ganar. 7. Dependiendo de tus decisiones al final del programa deberías recibir uno de los múltiples finales que hay.
Flujo alternativo	A. No hay un flujo alternativo para el programa.

4. DIAGRAMA CASO DE USO - ELIGE TU PROPIA HISTORIA

5. Diagrama casos de uso - Introduce contraseña para empezar la historia



7. CONCLUSIONES Y VALORACIÓN PERSONAL

El desarrollo de "La Rosa Encantada" ha supuesto un avance significativo en nuestra formación como programadores:

1. **Dominio del Lenguaje:** Hemos consolidado el uso de estructuras críticas en Java (switch, do-while, manejo de objetos String).
2. **Lógica Computacional:** La traducción de un diagrama de flujo narrativo a código ejecutable nos obligó a pensar en términos de estados y condiciones lógicas booleanas.
3. **Experiencia de Usuario:** Aprendimos que un buen programa no solo debe funcionar, sino ser comunicativo (uso de colores, mensajes de error claros).

El resultado final es un software robusto, libre de errores críticos de ejecución y que cumple con todos los requisitos narrativos y técnicos planteados inicialmente.

- **Buenas Prácticas de Programación:** Nos enfrentamos a la realidad de gestionar un archivo con cientos de líneas de código. Esto nos enseñó la importancia vital de comentar secciones complejas y utilizar una nomenclatura descriptiva (*camelCase*) para las variables, facilitando el trabajo en equipo y la revisión de errores.

7.1. Líneas de Trabajo Futuro (Mejoras Potenciales)

Aunque el proyecto cumple con los objetivos actuales, identificamos varias áreas donde el software podría escalar en futuras versiones:

1. **Persistencia de Datos (I/O):** Implementar las clases `FileReader` y `FileWriter` para permitir guardar y cargar la partida en un archivo de texto `.txt`, evitando que el progreso se pierda al cerrar la consola.
2. **Interfaz Gráfica (GUI):** Migrar de la interfaz de consola a una interfaz de ventanas utilizando la librería **Java Swing** o **JavaFX**, lo que permitiría visualizar imágenes reales de los personajes en lugar de arte ASCII.
3. **Refactorización a POO Pura:** Dividir el archivo monolítico `bellaBestia2.java` en múltiples clases (`Personaje.java`, `Enemigo.java`, `Juego.java`) para aplicar estrictamente el paradigma de Orientación a Objetos, mejorando la modularidad y seguridad del código.

8. BIBLIOGRAFÍA Y RECURSOS

Documentación Oficial:

Oracle Java SE Documentation (clases Scanner, Math, Random).

Herramientas de Diseño:

Draw.io para diagramado de flujo lógico.

Fuentes Literarias:

Adaptaciones varias del cuento "La Bella y la Bestia".

Asistencia Técnica:

Modelos de IA (OpenAI ChatGPT, Google Gemini) para consultas de sintaxis y depuración.